



Explore | Expand | Enrich

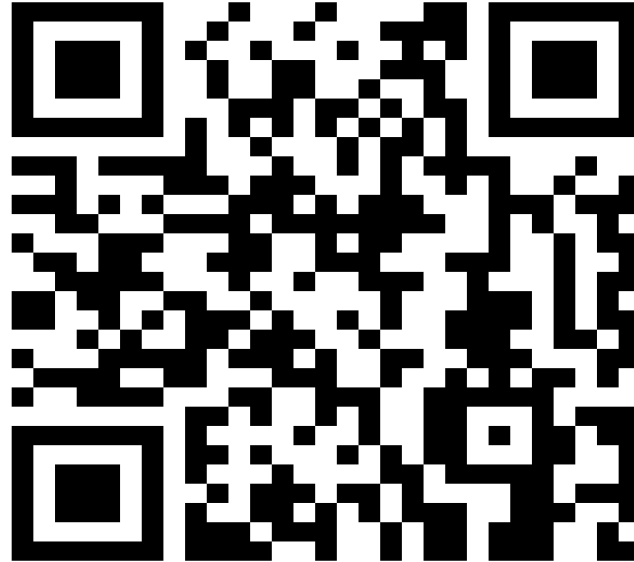
<Codemithra />TM

THE CELEBRITY PROBLEM



TEST TIME ON MINIMUM STACK

URL : <https://forms.gle/cqoa4QcjL8rPkzD8>

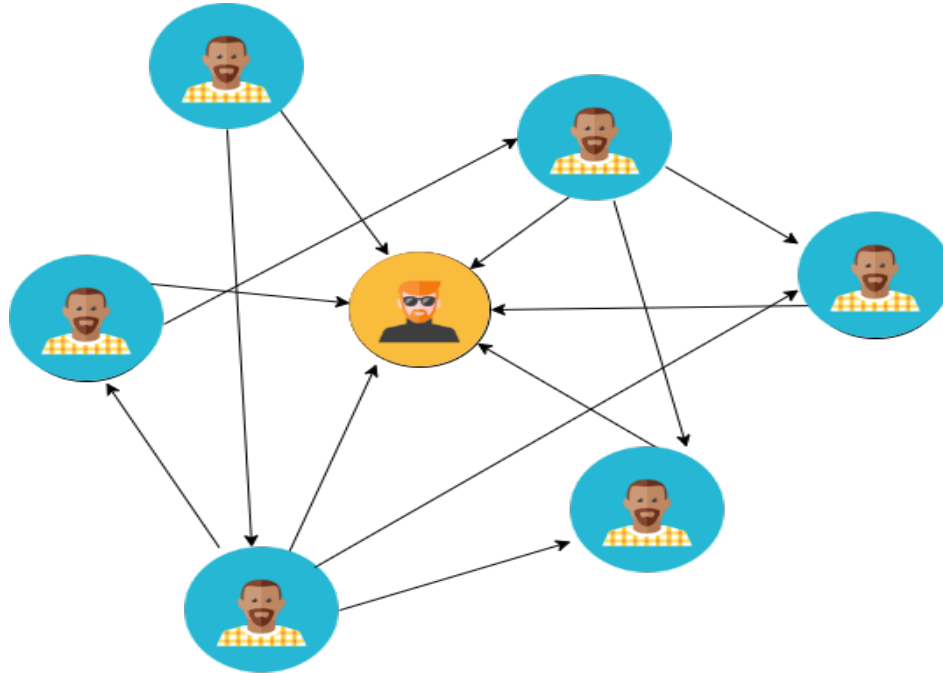


THE CELEBRITY PROBLEM

EXPLANATION

- The celebrity problem is identifying a celebrity in a group of people. In this context, a celebrity is defined as someone who is known by everyone in the group, but who does not know anyone else in the group.
- This problem can be used as an illustrative example for understanding algorithms and data structures.

THE CELEBRITY PROBLEM



THE CELEBRITY PROBLEM

EXPLANATION

- Given a group of people, each person may or may not know other people in the group.
- Your task is to identify if there is a celebrity in the group and, if so, who that celebrity is.
- A celebrity is someone who is known by everyone in the group but does not know anyone else.

THE CELEBRITY PROBLEM

ALGORITHM

To solve the celebrity problem, you can use a two-pass algorithm.

In the first pass, you find a potential celebrity candidate, and in the second pass, you verify if the candidate meets the celebrity criteria.

THE CELEBRITY PROBLEM

ALGORITHM

Step 1: Find a potential celebrity candidate.

Initialize two pointers, `left` and `right`, both initially pointing to the first person in the group.

Use a loop to iterate until `left` is not equal to `right`.

In each iteration, compare the knowledge of the two people pointed to by `left` and `right`.

If `left` knows `right`, set `left` to the next person; otherwise, set `right` to the next person.

At the end of this loop, `left` or `right` will be a potential celebrity candidate.

THE CELEBRITY PROBLEM

ALGORITHM

Step 2: Verify if the candidate is a celebrity.

Reset ``left`` to the beginning of the group, and set ``right`` to the candidate.

Iterate through the group again and check if everyone knows the candidate (i.e., ``left`` knows ``right`` for all pairs).

If the candidate is known by everyone and doesn't know anyone, they are the celebrity.

THE CELEBRITY PROBLEM

PSEUDOCODE

```
function findCelebrity(group):  
    left = 0  
    right = 0  
  
    // Step 1: Find a potential celebrity candidate  
    while left != right:  
        if group[left].knows(group[right]):  
            left = left + 1  
        else:  
            right = right + 1
```

THE CELEBRITY PROBLEM

PSEUDOCODE

```
// Step 2: Verify if the candidate is a celebrity
    right = candidate
    left = 0
    while left < length(group):
        if left != right and (group[left].knows(group[right]) or
!group[right].knows(group[left])):
            return "No celebrity found"
        left = left + 1
    return candidate
```

THE CELEBRITY PROBLEM

SAMPLE PROGRAM 1

INPUT

```
2          //n count
```

```
0 1
```

```
0 0
```

OUTPUT

```
Celebrity ID: 1
```

THE CELEBRITY PROBLEM

SAMPLE PROGRAM 1

INPUT

```
3
0 1 0
0 0 0
1 0 0
```

OUTPUT

No Celebrity

THE CELEBRITY PROBLEM

```
import java.io.*;
import java.util.*;
class Main {
    public static void main(String args[])
throws IOException {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    int[][] relationshipMatrix = new
int[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            relationshipMatrix[i][j] =
sc.nextInt();
        }
    }
    int celebrityId = new
Solution().findCelebrity(relationshipMatrix,
N);
    if (celebrityId != -1) {
```

```
System.out.println("Celebrity id: " +
celebrityId);
    } else {
        System.out.println("No
Celebrity");
    } } }
class Solution {
    int findCelebrity(int M[][], int n) {
        int candidate = 0;
        for (int i = 1; i < n; i++) {
            if (M[candidate][i] == 1) {
                candidate = i;
            }
        }
        for (int i = 0; i < n; i++) {
            if (i != candidate &&
(M[candidate][i] == 1 || M[i][candidate] ==
0)) {
                return -1;
            }
        }
        return candidate;
    } }
```

THE CELEBRITY PROBLEM

```
import java.util.*;
public class Main {
    public static int
    celebritySolution(int[][] mat) {
        Stack<Integer> stk = new Stack<>();
        for(int i=0;i<mat.length;i++) {
            stk.push(i);
        }
        while(stk.size() > 1) {
            int col = stk.pop();
            int row = stk.pop();
            if(mat[row][col] == 1) {
                stk.push(col);
            } else {
                stk.push(row);
            }
        }
        int x = stk.pop();
        for(int j=0;j<mat.length;j++) {
            if(j == x) continue;
            if(mat[x][j] == 1) {
```

```
                return -1;
            }
        }
        for(int i=0;i<mat.length;i++) {
            if(i == x) continue;
            if(mat[i][x] == 0) {
                return -1;
            }
        }
        return x;}
    public static void main(String[] args) {
        int[][] matrix = {{0,1,1,0},
                           {0,0,0,0},
                           {0,1,0,0},
                           {1,1,0,0}};
        int res = celebritySolution(matrix);
        if(res == -1) {
            System.out.println("There is no
            celebrity in the party");
        } else {
            System.out.println(res + " is the
            celebrity in the party");
        }
    }
}
```

THE CELEBRITY PROBLEM

TIME AND SPACE COMPLEXITY

Time complexity: $O(N)$, where N is the number of people in the group

Space complexity: $O(1)$ because the code uses a constant amount of additional space regardless of the input size, mainly for variables like candidate and loop counters.

INTERVIEW QUESTIONS

1. *Explain the Celebrity Problem*

Answer: The Celebrity Problem involves finding a celebrity in a group of people. A celebrity is someone who is known by everyone in the group but doesn't know anyone. It can be represented using a square matrix where $M[i][j]$ represents whether person i knows person j (1 for "knows" and 0 for "doesn't know"). The goal is to identify the celebrity efficiently.

INTERVIEW QUESTIONS

2. What is the time complexity of the stack-based solution for the Celebrity Problem?

Answer: The time complexity is $O(N)$, where N is the number of people in the group. The stack-based solution iteratively compares pairs of people, reducing the number of candidates until only one potential celebrity remains. This is done in linear time.

INTERVIEW QUESTIONS

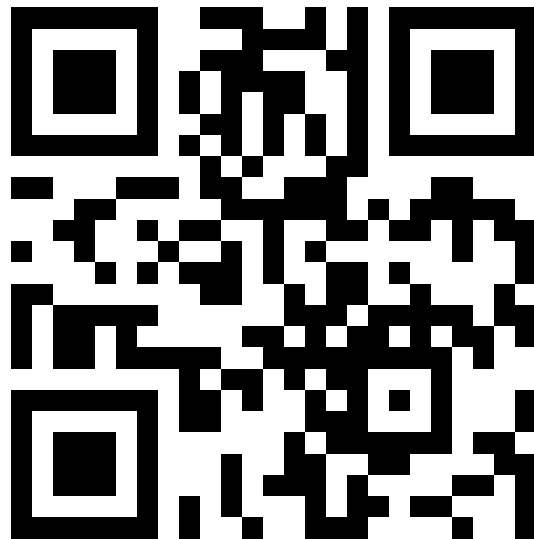
3. Can the Celebrity Problem be solved using a brute-force approach? What would be the time complexity?

Answer: Yes, the Celebrity Problem can be solved using a brute-force approach by checking each person's compatibility with the definition of a celebrity. However, the time complexity would be $O(N^2)$ because you would need to check every pair of people to determine if one knows the other.

INTERVIEW QUESTIONS

4. How does the Min Stack approach handle scenarios when the minimum element is popped from the main stack?

Answer: When the minimum element is popped from the main stack, the Min Stack approach ensures consistency by also popping the corresponding minimum element from the min stack. This guarantees that the top of the min stack always represents the minimum element present in the main stack at any given time.



<https://learn.codemithra.com>



THANK YOU



+91 78150 95095



codemithra@ethnus.com



www.codemithra.com